

✓ Generative AI & Transformers Discovery Project

Repository Name & Description

Repository Name: `Generative-AI-Transformers-Discovery`

Description (less than 300 characters): `Explore sentiment analysis, text generation, and summarization using Hugging Face Transformers. Learn pre-training vs. fine-tuning, and apply these models to a practical business workflow for Generative AI applications.`

✓ Discovery Phase: Setup and Initialization

First, we need to install the required libraries: `transformers` for accessing HuggingFace models and `torch` as its backend. We'll use `pip` for installation.

```
%%capture
!pip install transformers torch
```

Now that the libraries are installed, we can import `pipeline` from the `transformers` library and initialize the three distinct HuggingFace pipelines: `sentiment-analysis`, `text-generation`, and `summarization`.

```
from transformers import pipeline

# Initialize the sentiment-analysis pipeline
sentiment_pipeline = pipeline('sentiment-analysis')
print('Sentiment Analysis Pipeline Initialized:')
print(sentiment_pipeline.model)
print(sentiment_pipeline.tokenizer)

# Initialize the text-generation pipeline
text_generation_pipeline = pipeline('text-generation')
print('\nText Generation Pipeline Initialized:')
print(text_generation_pipeline.model)
print(text_generation_pipeline.tokenizer)

# Initialize the summarization pipeline
summarization_pipeline = pipeline('summarization')
print('\nSummarization Pipeline Initialized:')
print(summarization_pipeline.model)
print(summarization_pipeline.tokenizer)
```



```

[transformers] No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and rev
Using a pipeline without specifying a model name and revision in production is not recommended.
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:122: UserWarning:
Error while fetching 'HF_TOKEN' secret value from your vault: 'Requesting secret HF_TOKEN timed out. Secrets can only
You are not authenticated with the Hugging Face Hub in this notebook.
If the error persists, please let us know by opening an issue on GitHub (https://github.com/huggingface/huggingface\_h
warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits a
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_
config.json: 100%                               629/629 [00:00<00:00, 53.9kB/s]

model.safetensors: 100%                         268M/268M [00:06<00:00, 44.9MB/s]

Loading weights: 100%                           104/104 [00:00<00:00, 3867.23it/s]

tokenizer_config.json: 100%                     48.0/48.0 [00:00<00:00, 2.74kB/s]

vocab.txt: 100%                                232k/232k [00:00<00:00, 6.52MB/s]

[transformers] No model was supplied, defaulted to HuggingFaceTB/SmolLM3-3B and revision a07cc9a.
Using a pipeline without specifying a model name and revision in production is not recommended.
Sentiment Analysis Pipeline Initialized:
DistilBertForSequenceClassification(
  (distilbert): DistilBertModel(
    (embeddings): Embeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (transformer): Transformer(
      (layer): ModuleList(
        (0-5): 6 x TransformerBlock(
          (attention): DistilBertSelfAttention(
            (q_lin): Linear(in_features=768, out_features=768, bias=True)
            (k_lin): Linear(in_features=768, out_features=768, bias=True)
            (v_lin): Linear(in_features=768, out_features=768, bias=True)
            (out_lin): Linear(in_features=768, out_features=768, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (sa_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
          (ffn): FFN(
            (dropout): Dropout(p=0.1, inplace=False)
            (lin1): Linear(in_features=768, out_features=3072, bias=True)
            (lin2): Linear(in_features=3072, out_features=768, bias=True)
            (activation): GELUActivation()
          )
          (output_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        )
      )
    )
  )
  (pre_classifier): Linear(in_features=768, out_features=768, bias=True)
  (classifier): Linear(in_features=768, out_features=2, bias=True)
  (dropout): Dropout(p=0.2, inplace=False)
)
BertTokenizer(name_or_path='distilbert/distilbert-base-uncased-finetuned-sst-2-english', vocab_size=30522, model_max_
0: AddedToken("[PAD]", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
100: AddedToken("[UNK]", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
101: AddedToken("[CLS]", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
102: AddedToken("[SEP]", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
103: AddedToken("[MASK]", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
})
config.json: 100%                               1.92k/1.92k [00:00<00:00, 134kB/s]

model.safetensors.index.json: 100%               26.9k/26.9k [00:00<00:00, 1.66MB/s]

Download complete: 100%      6.15G/6.15G [02:08<00:00, 59.1MB/s]

Fetching 2 files: 100%                           2/2 [01:58<00:00, 118.05s/it]

Loading weights: 100%                           326/326 [00:00<00:00, 664.46it/s]

generation_config.json: 100%                     182/182 [00:00<00:00, 14.5kB/s]

tokenizer_config.json: 100%                     50.4k/50.4k [00:00<00:00, 4.47MB/s]

tokenizer.json: 100%                             17.2M/17.2M [00:00<00:00, 24.0MB/s]

special_tokens_map.json: 100%                   289/289 [00:00<00:00, 24.1kB/s]

chat_template.jinja: 100%                       5.60k/5.60k [00:00<00:00, 359kB/s]

Text Generation Pipeline Initialized:
SmolLM3ForCausalLM(
  (model): SmolLM3Model(

```

```

(embed_tokens): Embedding(128256, 2048, padding_idx=128004)
(layers): ModuleList(
  (0-35): 36 x SmolLM3DecoderLayer(
    (self_attn): SmolLM3Attention(
      (q_proj): Linear(in_features=2048, out_features=2048, bias=False)
      (k_proj): Linear(in_features=2048, out_features=512, bias=False)
      (v_proj): Linear(in_features=2048, out_features=512, bias=False)
      (o_proj): Linear(in_features=2048, out_features=2048, bias=False)
    )
    (mlp): SmolLM3MLP(
      (gate_proj): Linear(in_features=2048, out_features=11008, bias=False)
      (up_proj): Linear(in_features=2048, out_features=11008, bias=False)
      (down_proj): Linear(in_features=11008, out_features=2048, bias=False)
      (act_fn): SiLUActivation()
    )
    (input_layernorm): SmolLM3RMSNorm((2048,), eps=1e-06)
    (post_attention_layernorm): SmolLM3RMSNorm((2048,), eps=1e-06)
  )
  (norm): SmolLM3RMSNorm((2048,), eps=1e-06)
  (rotary_emb): SmolLM3RotaryEmbedding()
)
(lm_head): Linear(in_features=2048, out_features=128256, bias=False)
)
TokenizersBackend(name_or_path='HuggingFaceTB/SmolLM3-3B', vocab_size=128000, model_max_length=131072, padding_side='
128000: AddedToken("<|begin_of_text|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=T
128001: AddedToken("<|end_of_text|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=Tru
128002: AddedToken("<think>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=False),
128003: AddedToken("</think>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=False),
128004: AddedToken("<|finetune_right_pad_id|>", rstrip=False, lstrip=False, single_word=False, normalized=False, s
128005: AddedToken("<|reserved_special_token_2|>", rstrip=False, lstrip=False, single_word=False, normalized=False
128006: AddedToken("<|start_header_id|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special
128007: AddedToken("<|end_header_id|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=T
128008: AddedToken("<|eom_id|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=False),
128009: AddedToken("<|eot_id|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
128010: AddedToken("<|python_tag|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True
128011: AddedToken("<|im_start|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
128012: AddedToken("<|im_end|>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
128013: AddedToken("<tool_response>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=Fal
128014: AddedToken("</tool_response>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=Fa
128015: AddedToken("<tool_call>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=False),
128016: AddedToken("</tool_call>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=False)
128017: AddedToken("<code>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=False),
128018: AddedToken("</code>", rstrip=False, lstrip=False, single_word=False, normalized=False, special=False),
128019: AddedToken("<|reserved_special_token_11|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128020: AddedToken("<|reserved_special_token_12|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128021: AddedToken("<|reserved_special_token_13|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128022: AddedToken("<|reserved_special_token_14|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128023: AddedToken("<|reserved_special_token_15|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128024: AddedToken("<|reserved_special_token_16|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128025: AddedToken("<|reserved_special_token_17|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128026: AddedToken("<|reserved_special_token_18|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128027: AddedToken("<|reserved_special_token_19|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128028: AddedToken("<|reserved_special_token_20|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128029: AddedToken("<|reserved_special_token_21|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128030: AddedToken("<|reserved_special_token_22|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128031: AddedToken("<|reserved_special_token_23|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128032: AddedToken("<|reserved_special_token_24|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128033: AddedToken("<|reserved_special_token_25|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128034: AddedToken("<|reserved_special_token_26|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128035: AddedToken("<|reserved_special_token_27|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128036: AddedToken("<|reserved_special_token_28|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128037: AddedToken("<|reserved_special_token_29|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128038: AddedToken("<|reserved_special_token_30|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128039: AddedToken("<|reserved_special_token_31|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128040: AddedToken("<|reserved_special_token_32|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128041: AddedToken("<|reserved_special_token_33|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128042: AddedToken("<|reserved_special_token_34|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128043: AddedToken("<|reserved_special_token_35|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128044: AddedToken("<|reserved_special_token_36|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128045: AddedToken("<|reserved_special_token_37|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128046: AddedToken("<|reserved_special_token_38|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128047: AddedToken("<|reserved_special_token_39|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128048: AddedToken("<|reserved_special_token_40|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128049: AddedToken("<|reserved_special_token_41|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128050: AddedToken("<|reserved_special_token_42|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128051: AddedToken("<|reserved_special_token_43|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128052: AddedToken("<|reserved_special_token_44|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128053: AddedToken("<|reserved_special_token_45|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128054: AddedToken("<|reserved_special_token_46|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128055: AddedToken("<|reserved_special_token_47|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128056: AddedToken("<|reserved_special_token_48|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128057: AddedToken("<|reserved_special_token_49|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128058: AddedToken("<|reserved_special_token_50|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals
128059: AddedToken("<|reserved_special_token_51|>", rstrip=False, lstrip=False, single_word=False, normalized=Fals

```

[illegible]

[illegible]

128229: AddedToken("<|reserved_special_token_221|>", rstrip=False, lstrip=False, single_word=False, normalized=False)
 You can see the default model architectures loaded for each pipeline above. For example, sentiment-analysis often uses a BERT-like model, text-generation uses a GPT-like model (like GPT-2), and summarization might use models like BART or its. Each pipeline comes with its corresponding tokenizer.

128230: AddedToken("<|reserved_special_token_222|>", rstrip=False, lstrip=False, single_word=False, normalized=False)
 128231: AddedToken("<|reserved_special_token_223|>", rstrip=False, lstrip=False, single_word=False, normalized=False)
 128232: AddedToken("<|reserved_special_token_224|>", rstrip=False, lstrip=False, single_word=False, normalized=False)
 128233: AddedToken("<|reserved_special_token_225|>", rstrip=False, lstrip=False, single_word=False, normalized=False)
 128234: AddedToken("<|reserved_special_token_226|>", rstrip=False, lstrip=False, single_word=False, normalized=False)
 128235: AddedToken("<|reserved_special_token_227|>", rstrip=False, lstrip=False, single_word=False, normalized=False)
 128236: AddedToken("<|reserved_special_token_228|>", rstrip=False, lstrip=False, single_word=False, normalized=False)
 128237: AddedToken("<|reserved_special_token_229|>", rstrip=False, lstrip=False, single_word=False, normalized=False)
 128238: AddedToken("<|reserved_special_token_230|>", rstrip=False, lstrip=False, single_word=False, normalized=False)
 128239: AddedToken("<|reserved_special_token_231|>", rstrip=False, lstrip=False, single_word=False, normalized=False)
 128240: AddedToken("<|reserved_special_token_232|>", rstrip=False, lstrip=False, single_word=False, normalized=False)
 128241: AddedToken("<|reserved_special_token_233|>", rstrip=False, lstrip=False, single_word=False, normalized=False)
 128242: AddedToken("<|reserved_special_token_234|>", rstrip=False, lstrip=False, single_word=False, normalized=False)
 128243: AddedToken("<|reserved_special_token_235|>", rstrip=False, lstrip=False, single_word=False, normalized=False)
 128244: AddedToken("<|reserved_special_token_236|>", rstrip=False, lstrip=False, single_word=False, normalized=False)
 128245: AddedToken("<|reserved_special_token_237|>", rstrip=False, lstrip=False, single_word=False, normalized=False)

Technical Phase Implementation

Audit Task: Sentiment Analysis

We will now run an Audit by processing 5 test sentences through the sentiment-analysis pipeline. This includes a sarcastic example to evaluate the model's nuance detection capabilities.

```
test_sentences = [
    "I love this product, it's absolutely fantastic!",
    "This service is terrible, I'm extremely disappointed.",
    "The weather today is just delightful, isn't it? (said during a heavy storm)",
    "It was okay, nothing special.",
    "I'm so excited to get started on this project!"
]
```

```
print("--- Sentiment Analysis Audit Results ---")
for i, sentence in enumerate(test_sentences):
    result = sentiment_pipeline(sentence)
    print(f"\nSentence {i+1}: '{sentence}'")
    print(f"Sentence: {result[0]['label']} (Score: {result[0]['score']:.4f})")
```

```
15 # Initialize the summarization pipeline
--- Sentiment Analysis Audit Results
--> 16 summarization_pipeline = pipeline('summarization')
17 print('\nSummarization Pipeline Initialized:')
18 print(summarization_pipeline.model)
```

Sentence 1: 'I love this product, it's absolutely fantastic!'

Sentiment: POSITIVE (Score: 0.9999)

Sentence 2: 'This service is terrible, I'm extremely disappointed.'

Sentiment: NEGATIVE (Score: 0.9977)

Sentence 3: 'The weather today is just delightful, isn't it? (said during a heavy storm)'

Sentiment: POSITIVE (Score: 0.9999)

Sentence 4: 'It was okay, nothing special.'

Sentiment: NEGATIVE (Score: 0.9821)

KeyError: "Unknown task summarization, available tasks are ['any-to-any', 'audio-classification', 'automatic-speech-recognition', 'audio-question-answering', 'feature-extraction', 'fill-mask', 'image-classification', 'image-question-answering', 'image-segmentation', 'image-text-to-text', 'keypoint-matching', 'mask-generation', 'ner', 'object-detection', 'sentiment-analysis', 'table-question-answering', 'text-classification', 'text-generation', 'text-to-audio', 'text-to-speech', 'token-classification', 'video-classification', 'zero-shot-audio-classification', 'zero-shot-image-classification', 'zero-shot-object-detection']"

Creator Task: Text Generation

Now, we will execute the 'Creator' task using the text-generation pipeline. We'll use a domain-specific prompt related to customer support or a similar field and constrain the output to max_length=50 to observe the model's ability to generate concise text.

```
prompt = "In the context of customer support, a common issue is user password resets. When a customer requests a pas

print("--- Text Generation Creator Task ---")
# Fixing the length constraint: use max_new_tokens to ensure the desired length
generated_text = text_generation_pipeline(prompt, max_new_tokens=50, num_return_sequences=1, clean_up_tokenization_s

print(f"Prompt: '{prompt}'")
print(f"Generated Text: '{generated_text[0]['generated_text']}'")
```

[transformers] Passing `generation\_config` together with generation-related arguments=({'num_return_sequences', 'max_new_tokens', 'max_length'}) Both `max\_new\_tokens` (=50) and `max\_length` (=20) seem to have been set. `max\_new\_tokens` will take pr
 --- Text Generation Creator Task ---
 Prompt: 'In the context of customer support, a common issue is user password resets. When a customer requests a passw
 Generated Text: 'In the context of customer support, a common issue is user password resets. When a customer requests

Executive Summary Task: Summarization

For the 'Executive Summary' task, we will feed a 100-200 word news article into the summarization pipeline and evaluate the 20-30 word output for coherence and key point retention.

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import warnings
```

```
# Suppress UserWarning about HF_TOKEN if it's purely informational
warnings.filterwarnings("ignore", message="You are not authenticated with the Hugging Face Hub in this notebook.")
warnings.filterwarnings("ignore", message="You are sending unauthenticated requests to the HF Hub.")

model_name = 'sshleifer/distilbart-cnn-12-6'

# Explicitly load the summarization tokenizer and model
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

news_article = """Recent advancements in artificial intelligence have unveiled new possibilities for various industries. From healthcare to finance, AI is revolutionizing the way we live and work. The potential applications are vast and exciting, opening up new frontiers for innovation and progress. As technology continues to advance at a rapid pace, the future of AI holds immense promise for transforming our world in ways we can scarcely imagine today.""""

print("--- Summarization Executive Summary Task ---")

# Prepare the input for the model
inputs = tokenizer(news_article, return_tensors='pt', max_length=1024, truncation=True)

# Generate the summary using the model's generate method
# We need to explicitly set min_length and max_length for the desired 20-30 word output.
# A common heuristic is 1 token ~ 0.75 words, so 20-30 words would be around 27-40 tokens.
summary_ids = model.generate(inputs['input_ids'], min_length=20, max_length=40, num_beams=4, early_stopping=True, do_sample=False)
summary_text = tokenizer.decode(summary_ids[0], skip_special_tokens=True)

print(f"\nOriginal Article (Word Count: {len(news_article.split())}): \n{news_article}")
print(f"\nGenerated Summary (Word Count: {len(summary_text.split())}): \n{summary_text}")
```

```

Loading weights: 100%                               358/358 [00:00<00:00, 8441.33it/s]
--- Summarization Executive Summary Task ---

Original Article (Word Count: 152):
Recent advancements in artificial intelligence have unveiled new possibilities for various industries. Researchers at

Generated Summary (Word Count: 35):
Researchers at leading tech institutions have developed a novel AI model capable of predicting stock market fluctuat

```

- ✓ Action Phase

Pre-training vs. Fine-tuning Explanation

In this section, we will explain the conceptual difference between Pre-training (learning general language patterns) and Fine-tuning (adapting to specific tasks) in your own words, without using jargon.

Pre-training vs. Fine-tuning: An Analogy

Imagine you want to teach someone to become an expert chef.

- **Pre-training** is like sending them to a general culinary school. They learn fundamental cooking techniques, understand different ingredients, and get a broad knowledge of various cuisines. They become very good at understanding food in general, but they haven't specialized in anything specific yet.
- **Fine-tuning** is like sending that chef to a specialized restaurant after culinary school. For example, if they go to a high-end Italian restaurant, they will learn to apply their general cooking knowledge specifically to Italian dishes, perfecting pasta making, sauce preparation, and regional specialties. They adapt their broad skills to excel at a particular type of cooking.

In the world of AI:

- **Pre-training** involves training a large language model (LLM) on a massive amount of diverse text data (like the entire internet). During this phase, the model learns general language patterns, grammar, facts, common sense, and how words relate to each other. It becomes a general-purpose language understanding and generation engine.
- **Fine-tuning** involves taking that pre-trained model and training it further on a smaller, specific dataset tailored to a particular task. For instance, fine-tuning a model for sentiment analysis means training it on a dataset of text labeled with sentiments (positive/negative). The model adapts its general language understanding to become very good at *that specific task*.

Generative AI & Transformers Discovery Project

Table of Contents

- [Project Overview](#)
- [Installation](#)
- [Discovery Phase: Setup and Initialization](#)
- [Technical Phase: Implementation](#)
 - [Audit Task: Sentiment Analysis](#)
 - [Creator Task: Text Generation](#)
 - [Executive Summary Task: Summarization](#)
- [Action Phase](#)
 - [Pre-training vs. Fine-tuning Explanation](#)
 - [Business Workflow Design \(Upcoming\)](#)
 - [Hallucination & Bias Discussion \(Upcoming\)](#)
- [Usage](#)
- [Conclusion](#)

Project Overview

This project serves as a hands-on discovery journey into Generative AI and Transformer models, primarily utilizing the Hugging Face `transformers` library. The goal is to set up and experiment with core Natural Language Processing (NLP) tasks like sentiment analysis, text generation, and text summarization. We will explore model capabilities, identify limitations (e.g., sarcasm detection), and understand practical considerations like output length control. Furthermore, the project delves into fundamental AI concepts such as pre-training vs. fine-tuning and considers the broader implications of deploying Generative AI, including potential biases and ethical considerations.

Installation

To run this notebook, you need to install the `transformers` and `torch` libraries. The following command handles the installation:

```
!pip install transformers torch
```

Discovery Phase: Setup and Initialization

In this phase, we set up the environment and initialize the primary Hugging Face pipelines crucial for our exploration:

- **Sentiment Analysis Pipeline:** Used for classifying text into positive or negative sentiments.
- **Text Generation Pipeline:** Used for generating coherent and contextually relevant text based on a given prompt.
- **Summarization Pipeline:** Used for creating concise summaries of longer texts.

These pipelines automatically download and load default pre-trained models (e.g., DistilBERT for sentiment, GPT-2 for text generation, BART-CNN for summarization) and their corresponding tokenizers. Initial `UserWarning` messages regarding `HF_TOKEN` are noted but do not impede the functionality for public models.

Technical Phase: Implementation

Audit Task: Sentiment Analysis

We audited the sentiment analysis pipeline with various test sentences, including a sarcastic one. The model successfully identified straightforward positive and negative sentiments. However, it demonstrated a key limitation: it **failed to correctly detect sarcasm**, misclassifying a sarcastic negative statement as positive. This highlights the challenge of nuanced language understanding for current models.

Creator Task: Text Generation

The text generation pipeline was used to create domain-specific text based on a customer support prompt. We learned the importance of using `max_new_tokens` instead of `max_length` when attempting to control the length of the generated output, as `max_new_tokens` dictates the number of *new* tokens to generate beyond the prompt, ensuring the desired conciseness.

Executive Summary Task: Summarization

The summarization task involved generating a short executive summary from a news article. Initial attempts faced a persistent `KeyError` with the `pipeline` function. This was ultimately resolved by bypassing the `pipeline`'s task recognition and directly loading the

`AutoTokenizer` and `AutoModelForSeq2SeqLM` for the `sshleifer/distilbart-cnn-12-6` model. The model successfully generated a concise summary of the provided article, adhering to the specified word count constraints.

• • • • •